Walking Dog System Documentation

This manual explains how Walking Dog is assembled and how its main systems communicate. It is intended for developers who need to configure scenes, trace data, maintain gameplay and verify changes. Each system includes an overview, key state, method reference, execution flow and integration notes.

Revision: Development-JM at cdc8ebc. Documentation date: October 19, 2026.

1 Project Setup

Engine and dependencies

Open the `Walking Dog` folder as the Unity project. The recorded editor is Unity 6000.5.10f1. The enabled build order is `Title Screen` followed by `StepCounterTestAmar`. The latter scene contains the walking interface and feature screens.

Dependency	Repository configuration	Role
Unity Input System	1.20.0	Sensor/input integration and Back key handling
Unity UI	2.5.0	Canvas controls and TextMeshPro-based interfaces
Unity Test Framework	1.7.0	EditMode tests
Firebase Unity SDK	Imported under Assets/Firebase	Authentication and Firestore
Android native code	Assets/Plugins/Android	Google account picker, photo picker and walk location service
MapLibre and OpenFreeMap	WebView map implementation	Display map, route and territory overlays
Clipper2	Vendored under Scripts/ThirdParty	Territory polygon operations
Firebase Functions	Node.js 22 declared in functions/package.json	Backend source and emulator test tooling

Setup procedure

1. Open the project with its recorded Unity editor and Android build support.
2. Let Unity restore the committed package manifest and import project assets.
3. Review Firebase project configuration and Android signing fingerprints with the project owner. Follow `docs/google-sign-in.md` for the existing integration.
4. Open the title and gameplay scenes and check their enabled build order.
5. Check scene references for the persistent walk manager, Firebase bootstrap, map and feature controllers.
6. Validate rules against an emulator before a backend rollout. Test a fresh Android build on the devices being documented.

Project layout

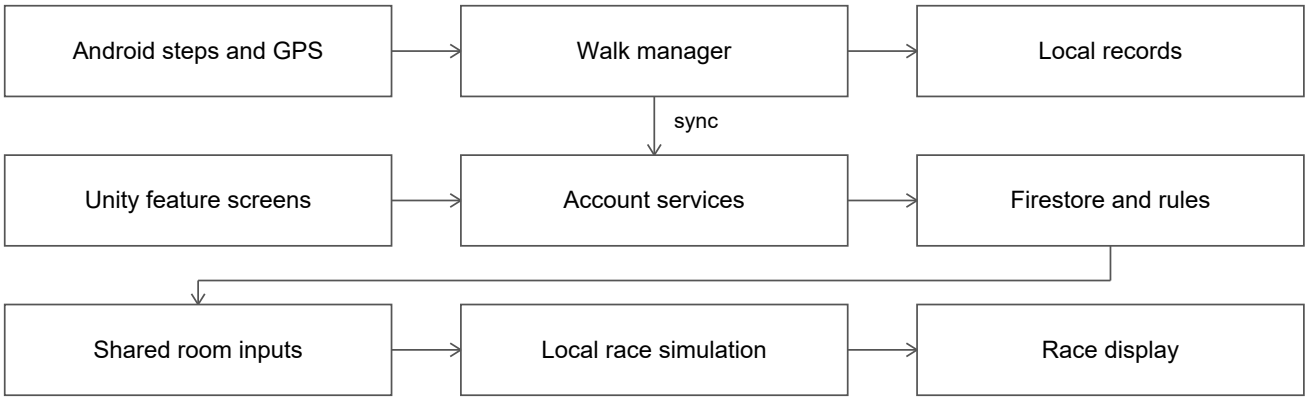
Location	Contents
Assets/Firebase	Account UI, authentication scripts and Firebase SDK
Assets/Scripts/Manager	Tracking coordinator, gacha and inventory UI
Assets/Scripts/Persistence	Local repositories, cloud adapters, wallet and continuous steps
Assets/Scripts/Racing	Dog stats, training, simulation, rooms and screens
Assets/Scripts/Leaderboard	Rankings, friends, profiles, following and shared routes
Assets/animation	Feature-specific animation folders and close helper
Assets/Tests/Editor	Unity automated test sources
functions, scripts, docs	Backend/test tools, map checks and documentation

2 Runtime Architecture and Shared Data

Overview and responsibilities

`StepCountAndGpsManager` coordinates activity, GPS walks, local walk records and territory access. `FirebaseWalkBootstrap` initializes account-dependent adapters and schedules synchronization. Feature controllers call their own services for purchases, training, rankings and rooms. `DogRaceSimulation` computes racing locally using captured inputs.

System relationships



Sensor input flows through the tracking coordinator into local state. Firebase synchronization uploads activity and summaries. UI services access Firestore for their account-owned or shared features. Shared room data supplies a frozen roster and seed to each client's local race simulation.

Key shared models

Model	Important data	Purpose
ContinuousSteps.Record	owner, streamId, total, acknowledged	Persistent installation activity and upload acknowledgement
SavedWalkSession	ID, owner, times, steps, distance, route samples, sync metadata	Durable complete or recoverable walk
WalkRoutePoint	Coordinates, sample time and segment metadata	Route quality and discontinuities
WalkSummary	ID, times, steps, accounting version, distance and duration	Cloud summary without coordinates
PointsWalletSnapshot	Balance, earned, spent and milestone claims	Validated immutable wallet view
GachaItem	ItemId, DisplayName, tier, Icon_img, CollarColorHex	Local catalog definition
DogStats	speed, stamina, acceleration, trainingCount	Saved dog progression and race input
RaceRoom / RaceEntrant	Room state, host, entrants and race inputs	Shared race coordination

Initialization flow

1. Account UI initializes Firebase and completes authentication.
2. The gameplay scene provides the active manager and its bootstrap component.
3. `InitializeAsync()` resolves Firebase dependencies, creates stores and binds account events.
4. Tracking writes local state; synchronization and feature requests use the current account.
5. Account changes invalidate affected pending UI work and reload account state.

Maintenance notes

`IsReady` on the bootstrap means initialization succeeded. It does not prove the player is signed in or that the server is reachable. A preview such as `StepPoints` is not a spendable wallet balance. Use each feature's validated model rather than deriving cloud values from a label.

3 Authentication System

Overview

`FirebaseLoginManager` handles email/password registration and sign-in, Google authentication and sign-out. `LoginUIManager` manages title-screen input, request feedback and successful-login navigation. Android Google sign-in uses `AndroidGoogleSignIn` and `GoogleSignInBridge.java` before exchanging the token with Firebase.

Key state and events

Member	Meaning
Instance	Shared FirebaseLoginManager reference
IsFirebaseReady	Dependency initialization status
InitializationError	Initialization failure text
IsGoogleLoginInProgress	Google request already active
FirebaseAuth current user	Identity used by cloud adapters

Method reference

Method	Responsibility
LoginUser(email, password)	Submit an email/password sign-in request
RegisterUser(email, password)	Create an account using the supplied credentials
LoginWithGoogle()	Request Android Google credentials and exchange them with Firebase
SignOutUser()	Sign out and request Google credential-session cleanup
GetCurrentUser()	Read the current Firebase user
LoginUIManager.loadNextScene(sceneName)	Apply the scene transition used after successful authentication

Execution flow

1. A title-screen button validates inputs and starts its authentication path.
2. Firebase returns success or a failure; the UI restores controls and presents feedback.
3. Google sign-in additionally passes through the native Android account picker.
4. Only a successful Firebase exchange enters the authenticated game flow.
5. Signing out triggers the account-state handling used by dependent systems.

Configuration and dependencies

Inspect `Assets/Firebase/LoginUIManager.cs`, `FirebaseLoginManager.cs`, `AndroidGoogleSignIn.cs` and `Assets/Plugins/Android/GoogleSignInBridge.java`. The configured Android package, Firebase configuration and signing certificates must agree. An imported desktop/editor project cannot demonstrate that Android Google sign-in works.

Integration notes

Keep tokens in the authentication path; do not persist or print them for documentation. Google sign-in is Android-specific and reports unsupported-platform feedback elsewhere. Cancellation and failure should restore the login UI. The existing native flow has a two-minute timeout. Capture the actual title-screen wiring and its success/error states for the final handover.

4 Continuous Step Tracking System

Overview and responsibilities

`StepCounterDisplay` integrates Unity/Android sensor input and motion filtering. It reports the current sensor-session count to `StepCountAndGpsManager.SetStep()`. `ContinuousSteps` converts that observed count into durable, account-scoped increments. Walk boundaries do not reset continuous totals.

Key fields and properties

Member	Type or role
ContinuousSteps.Record.owner	Account ID; empty for guest activity
streamId	Persistent UUID identifying this installation stream
total	Cumulative tracked steps in the stream
acknowledged	Highest locally acknowledged uploaded total
highWater	Sensor count already observed; prevents re-crediting corrections
TotalTrackedSteps	Manager property for the active local account total
StepTrackingStatus / Error	Local saving and tracking feedback

Method reference

Method	Responsibility
BeginStepSensorSource()	Manager entry point for beginning a new sensor source
SetStep(steps)	Observe sensor steps, update session progress and check milestones
ContinuousSteps. Observe(owner, sensorSteps, legacyWalk)	Add newly observed activity and save it
Get(owner)	Load/create an owner-specific record, using backup recovery
Flush(owner)	Persist the current record
Snapshot(owner)	Persist and copy the record before upload
Acknowledge(uploaded)	Advance the saved acknowledgement after successful upload

Data flow

1. Sensor count increases; only the increment above the observed high-water count is considered.
2. The increment belongs to the active owner. Switching accounts does not transfer earlier activity.
3. The updated total is saved under `WalkSessions/ContinuousSteps`.
4. The bootstrap submits the absolute total roughly every 15 seconds while running.
5. The Firestore transaction credits only newly observed steps and crossed ten-step point thresholds.

6. A successful response advances the local acknowledgement. A retry of the same total does not earn again.

Important behavior

The main total is not a daily counter or an imported all-time health total. Ten new credited steps earn one point, with remainders carried forward. Guest activity remains local. The native background walk service records GPS; continuous steps do not establish a separate always-running step service. Sensor behavior while paused, screen off or force-stopped requires physical-device verification.

5 GPS Walk Recording System

Overview

`GPSTracker` handles foreground location polling. `StepCountAndGpsManager` records active walks and routes. `WalkGpsFilter` validates sample quality and marks discontinuities. `AndroidWalkTracking` bridges the active walk to `WalkLocationService.java`, which records native GPS samples while Unity is paused.

Key properties and configuration

Member or threshold	Meaning
HasFreshLocation	Accurate fix whose timestamp remains fresh
IsWalkingSessionActive	Recording session is active
WalkingSessionSteps	Steps recorded within this walk
WalkingSessionDistanceMeters / WalkingSessionDurationSeconds	Walk-specific distance and active duration
HasTrackingGaps	Route contains interruptions
AccurateGpsThresholdMeters	10 metres
FreshSeconds / GapSeconds	10-second freshness and 15-second gap thresholds
BackgroundTrackingStatus	Native route recording or foreground fallback explanation

Method reference

Method	Responsibility
BeginWalkingSession(clearExistingRoute)	Capture owner/start state and begin route/background recording
EndWalkingSession()	Stop and drain tracking, then finalize/save the walk
SetGpsLocation(...)	Ingest foreground fixes through the manager
WalkGpsFilter. Accept(..., out startsSegment)	Validate the fix and decide whether it becomes a route sample
InterruptTracking(reason)	Invalidate current location and mark interrupted continuity
GetRoutePointSamplesSnapshot()	Return route samples for downstream use

Execution flow and filtering

1. A precise fresh fix enables starting a walk; the manager records account ownership.
2. During native tracking, the manager consumes buffered native samples rather than duplicating foreground route recording.
3. Coordinates, accuracy and timestamps are checked. Stale, future, duplicate or out-of-order samples are rejected as appropriate.
4. Movement beyond 6 metres/second plus a 5-metre allowance is treated as a jump. Small movement below max(3 metres, 0.75 times accuracy) does not add distance.
5. Interruptions start a new route segment. Display and distance calculation must preserve that break.
6. Finishing drains native samples, saves the completed record and presents its summary.

Integration notes

Do not join samples across gaps or evaluate territory from downsampled display coordinates. The native service operates only during an active walk and does not restart after force-stop. If permissions, storage or service startup fail, the UI reports foreground-only fallback. Inspect the actual saved route alongside the displayed map during verification.

6 Persistence Recovery and Walk History

Overview

`LocalWalkRepository` stores completed walks and mutable recovery checkpoints. `WalkSyncService` uploads eligible account-owned summaries. `WalkRecoveryUI` provides resume/save-and-finish and summary actions. `WalkHistoryUI` combines history loading, route display and navigation.

Key storage and state

Location or model	Purpose
Application.persistentDataPath/WalkSessions	Local walk storage root
WalkSessions/Active	Mutable recovery checkpoints
WalkSessions/Background	Native sample journal
WalkSyncMetadata	state, attempts, nextAttemptUtc, syncedAtUtc
WalkUploadState	Pending, RetryNeeded or Synced
RecoverableWalk	Owner-matching checkpoint without a completed local record

Method reference

Method	Responsibility
LocalWalkRepository.Save(walk)	Validate and persist a completed walk
TryLoad(path, out walk) / LoadAll()	Load stored walk records
AssignOwner(id, ownerUserId)	Explicitly assign an eligible local walk
SaveCheckpoint()	Save active-session recovery state
RecoverWalk(continueWalking)	Resume or finalize the recoverable walk
SaveCurrentWalkingSession()	Save/retry saving the completed session
WalkSyncService.SyncPendingAsync(token)	Upload unsynced walks with durable retry metadata
WalkHistoryUI.Open() / Close()	Open history or restore its suspended map

Execution flow

1. Start, periodic updates, pause and completion produce checkpoints as applicable.
2. Atomic file replacement and backups protect persisted records.
3. Recovery matches the original owner, replays relevant native samples and inserts a break before resumed recording.
4. A completed file takes priority over an older active checkpoint.
5. Upload processing persists retry metadata first, uploads a coordinate-free `WalkSummary`, then marks it synced after acknowledgement.
6. Failed uploads remain retryable with increasing delay. Another account's local walks are skipped.

Dependencies and notes

`IWalkCloudStore` defines owner identity and acknowledged uploads; `FirebaseWalkCloudStore` implements the adapter. `FirebaseWalkHistoryStore` supports cloud history. Full coordinates use a separate sharing path. An unsaved completed walk must not be silently replaced by a new one. Automatic sign-in does not imply permission to adopt guest records.

7 Territory Capture System

Overview and responsibilities

`TerritoryCapture` evaluates eligible saved routes. `TerritoryGeometry` and vendored Clipper2 perform polygon operations. `LocalTerritoryRepository` records ownership and processed walk receipts. `TerritoryService` exposes current-owner geometry and summary text to the map and UI.

Rules and key state

Constant or state	Value or responsibility
Version	2, polygon trail/loop claims
MinimumTrailMeters	50 metres
TrailRadiusMeters	5 projected metres on each side
MinimumLoopMeters	200 metres
ClosureMeters	30 metres from final accepted sample to start
Minimum loop area	2,500 square metres
TerritoryService.Polygons / Revision	Owner geometry and overlay change marker

Method reference

Method	Responsibility
TerritoryCapture.Evaluate(walk, fullLoop)	Select polygon trail/loop evaluation when true; retain the legacy tile path when false
EvaluateLoop(walk)	Wrapper passing true for current polygon trail/loop evaluation
TerritoryService. Refresh(authenticatedOwner)	Load owner ledger and replay applicable completed walks
TerritoryService. Summary(walkId, walkOwner, unsaved)	Explain claim/save outcome for the selected session

Execution flow

1. An account-owned completed walk supplies its full route and quality metadata.
2. Eligibility and route continuity are checked before geometry is accepted.
3. A qualifying trail produces a buffered corridor. A valid closed loop can also fill its interior.
4. The ledger unions new regions with existing ownership and computes newly added area.
5. The walk ID and area receipt are committed together. Replaying that walk returns its existing result.
6. Map state reads the current owner's polygons and revision to refresh its overlay.

Dependencies and limits

Self-crossing, touching or retraced non-adjacent edges prevent interior filling but may leave a valid corridor. Overlapping owned land counts once. Projected buffer width differs from ground width with latitude. Processing is bounded to 6,000 samples; extreme latitudes and date-line crossings are unsupported. Failed local claims are retried by the manager, and owner changes clear the prior overlay. Territory remains a local ledger in this implementation; it is not documented as a shared cloud land contest.

8 Map Display System

Overview

`OpenFreeMapWebViewMap` creates and manages the Android WebView map. It sends position, route, equipment and territory state to its web content. `MapSuspender` and individual feature controllers use owner-based visibility blocks while Unity overlays are open.

Key state and configuration

Item	Purpose
RectTransform on map object	Determines native screen bounds
Map style and zoom	Select presentation and walking-scale view
visibilityBlockers	Tracks every object requesting map suspension
Route segment metadata	Prevents lines spanning GPS gaps
Territory revision	Detects changed ownership geometry
EquippedGachaMarkerIcon	Encodes equipped artwork for WebView use

Method reference

Method	Responsibility
Suspend(owner)	Add an owner's visibility block
Resume(owner)	Release that owner's block
ForceSync()	Request an immediate map-state refresh
SetZoom(newZoom)	Change the zoom configuration
BuildMapState() [private]	Build the payload from current game state
CreateOrShowWebView() [private]	Create/attach or show the native view when ready
MapSuspender.OnEnable / OnDisable	Apply/release blocks for an overlay's lifetime

Lifecycle and flow

1. The component waits for suitable layout bounds before creating the native view.
2. It loads map web content and synchronizes the current state.
3. GPS updates, equipment changes and territory revisions refresh the map payload.
4. Opening a feature overlay blocks the map under that owner's reference.
5. Closing releases only that reference. Other active overlays can keep the map hidden.
6. Pause/resume and layout changes refresh attachment, size and visibility as needed.

Integration notes

A fallback map location before GPS is available is distinct from a usable tracking fix. The editor cannot establish Android WebView correctness. Keep every Suspend/Resume owner consistent and release it during cleanup. Route previews must fit the saved route rather than assume the current live location. Inspect both vector and fallback rendering and preserve map-data attribution.

9 Wallet and Reward Accounting System

Overview

`PointsWalletSnapshot` validates immutable totals and reward calculations. `PointsWalletSession` owns account-sensitive loading, request state and spend/claim actions. `FirebasePointsWalletStore` implements Firestore wallet operations; `FirebaseContinuousStepsStore` applies current activity earnings.

Key data

Member	Meaning
Balance	TotalEarned minus TotalSpent
TotalEarned / TotalSpent	Lifetime accounting totals
MilestoneRollsClaimed	Number of free milestone rolls already consumed
StepsPerMilestoneRoll	10,000 step-equivalents
DisplayText	Wallet value/status intended for UI
Owner / HasPendingWalks	Active account and pending synchronization state

Method reference

Method	Responsibility
PointsWalletSnapshot.RewardForSteps(steps)	Ten-step conversion used by legacy per-walk accounting
MilestoneRollsForEarnedPoints(earned)	Derive free-roll entitlement from lifetime earned points
PointsWalletSession.SynchronizeAccount()	Detect and reset for a changed account
RefreshAsync()	Load/reconcile the wallet
SpendPointsAsync(amount, receiptId)	Submit an identified spend
ClaimMilestoneRollAsync(receiptId)	Submit an identified free-roll claim
FirebaseContinuousStepsStore. UploadAsync(...)	Atomically credit new continuous steps and related totals

Current reward flow

1. Upload an absolute stream total with its stable stream ID.
2. Read the stream cursor, account activity, wallet and ranking document.
3. Credit only newly observed activity; award newly crossed ten-step thresholds.

4. Commit the linked updates in one transaction according to the rules.
5. Refresh the confirmed wallet view. Retrying an already credited total adds no earnings.

Notes and dependencies

Current activity carries step remainders across updates and walks. Legacy walk receipts retain their older accounting path. `StepPoints`, `WalkingSessionPoints` and `getPoint()` are previews; purchasing must use the saved wallet. Native requests can finish after the UI stops waiting, so stable receipts/cursors matter. A pending balance is not fabricated as zero before a successful load. Rules validate accounting relationships, not the physical truth of client sensor measurements.

10 Gacha and Collar Inventory System

Overview

`GachaManager` selects a collar from its local catalog and coordinates paid or milestone pulls. `CollarInventoryStore` persists unlocked/equipped item IDs, and `CollarInventoryUI` creates cards for the catalog. `GachaInventoryCardUI` binds individual card presentation and equip callbacks.

Inspector and model reference

Field	Current role or saved scene value
itemCatalog	GachaItem assets; inventory and pull UI should resolve the same IDs
pullCost	100 points
bronzeRate / silverRate / goldRate	Saved values 0.70 / 0.25 / 0.05
GachaItem.ItemId / DisplayName	Stable saved identity and presentation name
tier / Icon_img / CollarColorHex	Catalog rarity and appearance
content / itemCardPrefab	Inventory card parent and prefab references

Method reference

Method	Responsibility
OnGachaButtonClicked()	Validate wallet/catalog, spend, update ranking balance, then grant
OnMilestoneButtonClicked()	Consume an earned free roll, then grant an item
RollItem() [private]	Choose Gold, Silver or Bronze, then an item within that tier
UnlockItemAsync(uid, item)	Cache/equip the item and persist inventory/profile data
EquipItemAsync(uid, item)	Save the choice of an already unlocked item
SyncFromCloudToLocalAsync(uid)	Restore remote inventory IDs into the local cache
CollarInventoryUI.RefreshInventory()	Rebuild cards and their locked/unlocked states

Execution flow

1. Resolve the current owner and wallet; reject an invalid catalog or unavailable funds.
2. Select a tier and choose uniformly from configured items in that tier.
3. Submit the debit or milestone claim. Paid pulls also synchronize the ranking balance.
4. Grant the item, update local equipped appearance and persist inventory/profile state.
5. Refresh the result label and map; re-enable controls after completion or failure.

Important notes

`bronzeRate` is serialized but `RollItem()` chooses Bronze as the remaining probability after Gold and Silver. Keep the configured rates consistent. IDs must remain stable after players own items. Unlocking currently equips the item; repeated IDs remain a set of unlocks rather than counted duplicates. Selection occurs on the client. Debit/claim and grant are separate operations, so a failure after payment can leave the reward ungranted. Do not describe this flow as an atomic purchase or as server-verified randomness.

11 Dog Training System

Overview

`DogStats` contains progression rules shared by training and simulation. `FirebaseDogTrainingStore` loads/creates the account dog and atomically purchases a stat upgrade. `DogGameScreens` binds the stat rows, wallet feedback and pending receipt retry behavior.

Data and constants

Member	Value or meaning
speed / stamina / acceleration	Integer dog attributes
StartingValue	40 for each baseline stat
MaximumValue	100
TrainingGain	5 to one selected stat
TrainingCost	20 wallet points
trainingCount	Number of applied training sessions
pendingTraining	Receipt ID and selected stat saved for uncertain acknowledgements

Method reference

Method	Responsibility
DogStats.Copy()	Return a separate stat snapshot
DogStats.Train(stat, balance)	Validate the purchase and return upgraded stats
DogStats.Validate()	Check stat and count bounds
FirebaseDogTrainingStore. LoadAsync(owner, token)	Read the account dog or create its baseline
TrainAsync(owner, stat, receiptId, token)	Debit wallet, write upgraded dog and receipt together
DogGameScreens. TrainSpeed / TrainStamina / TrainAcceleration	UI entry points
RefreshTraining()	Reload data and retry a pending identified training purchase

Transaction flow

1. Load the authenticated owner's dog and wallet.
2. Save a training receipt ID before submission.
3. In one Firestore transaction, read dog, wallet and receipt.
4. An existing receipt returns the saved result rather than charging again.
5. Otherwise validate funds/stat cap, debit 20 points, add 5 to the stat and write the receipt.
6. Refresh UI and request ranking balance synchronization after acknowledgement.

Configuration and dependencies

Training requires signed-in cloud state. Signed-out practice uses a baseline dog without saved upgrades. The stat/price contract is enforced in both `DogStats` and `firestore.rules`; changing only the UI is insufficient. Account switches invalidate displayed requests and reload the new account's dog. Inspect `StatRow` value/bar/train references and test both insufficient funds and cap behavior.

12 Race Simulation System

Overview

`DogRaceSimulation` is independent of Unity and Firebase. It receives copied entrants and a seed, advances fixed ticks and calculates finish crossings. Rendering reads runners and standings; the rendered frame rate does not choose the winner.

Key models and properties

Member	Meaning
DogRaceEntry	ID, name, copied stats and equipped item ID
DogRaceRunner	Entry, distance, speed, finish time and seeded form
TickSeconds / DefaultLength	0.05 seconds / 200 metres
Runners / Elapsed / Completed	Simulation state for UI
FinishTime	Interpolated crossing time; infinity before finishing

Method reference

Method	Responsibility
DogRaceSimulation(entries, seed, length)	Validate inputs and initialize seeded form per entrant
Advance(seconds)	Consume accumulated time through fixed simulation ticks
Standings()	Order by finish time, then distance and stable entrant ID
PracticeField(player, equippedItemId)	Build the player plus Dash, Pip and Scout practice lineup
DogGameScreens.StartRace()	Start a practice simulation using the current dog snapshot
RenderRace() [internal]	Update marker positions, summaries and result text

Simulation flow

1. Copy each entrant's stats and equipment so later changes cannot alter the race.
2. Seed a small form multiplier in a stable ID order.
3. Each tick calculates fatigue, target speed, acceleration and distance.
4. Interpolate a finishing runner's crossing within the tick.
5. Render progress and final standings from simulation data.

Usage example

```
var field = DogRaceSimulation.PracticeField(new DogStats());
var simulation = new DogRaceSimulation(field, seed: 1234);
simulation.Advance(1.0); // Advance one second, not a whole race.
var currentOrder = simulation.Standings();
```

Dependencies and notes

The constructor permits two to eight distinct entrants; the current room/UI feature uses four lanes. Speed controls cruising pace, Acceleration controls reaching it, and Stamina delays fatigue. The practice partners are simulated. There are no current race fees, rewards or persistent competition-result records in this phase. Use the same entrant inputs and seed when comparing clients.

13 Multiplayer Race Room System

Overview

`FirebaseRaceLobbyService` coordinates Firestore rooms. `RaceRoom` and `RaceEntrant` represent their validated state. `DogGameScreens` manages commands, errors, listeners and transition into a local race. This shares race inputs rather than streaming every runner position.

Key state and configuration

Member	Purpose
RaceRoomCodes	Five-character room-code creation, formatting and normalization
currentRoom	Active room snapshot
roomSubscription	Disposable Firestore listener
LobbyTimeout	30-second UI request wait
roomRevision / lobbyCommandVersion	Reject stale listener/command work
roomCodeInput / lobbyEntrantRows	Serialized code entry and roster UI

Method reference

Method	Responsibility
CreateRoomAsync(...)	Create a host room using saved entrant data
JoinRoomAsync(...)	Add the caller to an eligible room transactionally
LeaveRoomAsync(...)	Remove a waiting guest or close a host-owned room
StartRaceAsync(roomCode, hostUid, seed, token)	Freeze room inputs and start from the host
ListenRoom(roomCode, onUpdate, onError)	Subscribe to room updates; returns a disposable handle
RaceRoom.BuildSimulationField()	Convert entrants and fill unused lanes with simulated dogs
DogGameScreens. CreateRoom / JoinRoom / StartLobbyRace / LeaveRoom	Scene button entry points

Execution flow

1. A signed-in host creates a room and shares its code.
2. Joining validates capacity and reads the caller's saved dog/equipment data.
3. Clients listen to the same room and display the entrant roster.
4. The host starts; rules restrict the change and freeze the roster.
5. Each client builds the same race field and seed and starts its local simulation.
6. Closing, account changes or teardown stop listeners and invalidate obsolete callbacks.

Integration notes

At most four players join; simulated dogs fill unused lanes. Room listing is denied; an authenticated user with a code may read that room. Creation metadata must survive joins/leaves. Requests restore controls after timeout/error, but native operations may complete late. Host departure closes a waiting room; host disconnection has no automatic expiry in this phase. Test on two updated clients and compare both entrants and finishing order.

14 Rankings Friends and Profile System

Overview

`FirebaseLeaderboardService` supplies rankings and implements profile/friend operations through its partial-class files. `FirebaseLeaderboardWriter` synchronizes aggregates. `LeaderboardPanelUI`, `FriendsPanelUI` and profile-photo components bind display and actions.

Key data

Model or setting	Role
LeaderboardMetric	Distance or Steps
LeaderboardScope	Global or Friends
LeaderboardSnapshot	Entries, current player, scope and profile photo
FriendEntry	Player ID, display name, requester, accepted flag and photo
FriendAction	Send, Accept or Remove
FriendCodes	Eight-character codes, displayed with a central hyphen

Method reference

Method	Responsibility
FirebaseLeaderboardService. CreateAsync(bootstrap)	Create an account-aware service after initialization
LoadAsync(metric, scope, token)	Fetch and rank the selected scope/metric
SaveDisplayNameAsync(displayName, token)	Validate and persist a nickname
GetFriendCodeAsync(token)	Reserve/read the account's permanent friend code
LoadFriendsAsync(token)	Read requests and accepted connections
ChangeFriendAsync(code, action, token)	Send, accept or remove a relationship
SaveProfilePhotoAsync(photo, token)	Store/reset custom profile photo data

Data flow

1. The panel resolves the current service and requested metric/scope.
2. Global returns its top 50; Friends loads the player and accepted friends directly.
3. Entries and photos bind into cards, with the current player's summary handled separately.

4. Friend management loads requests or submits the selected relationship action.
5. Refresh/reopening retrieves later changes; ordinary friend management does not use a real-time listener.

Integration notes

Players without counted activity do not receive a fabricated rank. The Friends tab changes ranking scope, not global score privacy. Accepted friends and followers are separate models. Profile photo selection uses an Android bridge; a reset can use the Google photo or initials. Some inline comments refer to older friend-code lengths; the model implementation creates eight characters. Verify UI wiring and account invalidation rather than relying on those comments.

15 Following Activity and Shared Routes

Overview

The social partial service, `SocialPanelUI`, `SocialWalkDetailsUI` and `SocialRouteSync` implement following and optional walk visibility. Following is directional. A person must opt into sharing before followers can read their walk summaries and uploaded routes.

Key data and responsibilities

Data	Purpose
socialProfiles/{uid}.shareActivity	Explicit activity-sharing setting; missing means private
social/{follower}/following/{followed}	Directional follow index
social/{followed}/followers/{follower}	Atomic mirrored index
users/{uid}/walks/{walkId}	Walk summaries, read under applicable permissions
users/{uid}/sharedRoutes/{walkId}	Separately uploaded route coordinates
Local saved walk	Owner fallback when its original recording still exists

Method reference

Method	Responsibility
SetFollowingAsync(target, following, token)	Create/remove the directional mirrored relationship
SetActivitySharingAsync(sharing, token)	Change the active account's sharing preference
LoadSocialAsync(tab, target, token)	Load the requested profile, relationship or activity view
FirebaseWalkBootstrap. RequestRouteSharingSync()	Schedule a route synchronization pass
SocialRouteSync	Find eligible saved local routes and coordinate uploads
SocialWalkDetailsUI	Display selected walk statistics and available route

Sharing flow

1. A player opts in on Profile; opening Profile alone does not enable sharing.

2. Existing and future eligible local routes can upload after their summaries synchronize.
3. A follower loads activity, then opens a walk's details and route.
4. Unfollowing or disabling sharing denies subsequent follower reads.
5. The owner may still use an original local recording when remote route loading fails.

Lifecycle and limits

Activity includes the owner's walks even with sharing off. Routes are not embedded in ordinary summary uploads. The bootstrap schedules upload checks at startup/resume, after relevant changes and periodically while running. Disabling sharing stops new route uploads and denies new follower reads; it does not erase relationships or revoke copies already viewed. If an unuploaded route's local recording is gone, its coordinates cannot be rebuilt from the summary. Preserve the detail-popup navigation and list position when returning to activity.

16 Screen Navigation Animation and Notifications

Overview

Feature controllers own screen state, request lifetimes and map visibility. `DogGameScreens` coordinates Training, Race and Lobby. `DisableanimatedObject` receives animation events and disables their target on the following frame. `LocalPhoneNotifications` handles milestone notices; tracking notifications are also part of the native walk service.

Key Inspector references

Field	Purpose
trainingScreen / raceScreen / lobbyScreen	Feature roots controlled by DogGameScreens
stats / lanes	Stat labels, sliders, training buttons and runner markers
trainingAnimator / raceAnimator / lobbyAnimator	Optional explicit close animators; root Animator fallback
trainingCloseTrigger / raceCloseTrigger / lobbyCloseTrigger	Defaults to onClose
DisableanimatedObject.disableTarget	Optional object to disable instead of the helper object
closing / closingScreen	Track an in-progress exit and its completion target

Method reference

Method	Responsibility
OpenTraining / OpenRace / OpenLobby	Navigate into the selected feature
Close()	Cancel current work, leave the room and begin animated close
FinishCloseAnimation()	Disable screens and release map blocks
NotifyScreenDisabled(screen)	Receive the matching animation helper's completion
DisableanimatedObject.setActiveFalse()	Schedule deferred notification and target deactivation
LocalPhoneNotifications.Initialize()	Prepare notification support/permission
NotifyNewMilestone(accountId, totalFreeRolls, availableRolls)	Notify newly reached milestones without repeating old ones

Close sequence

1. Cancel/invalidate current requests and stop the displayed race/room state.
2. Find the active screen's Animator and trigger its close state if configured.
3. The new Race/Training close clips call `setActiveFalse` at 0.5 seconds.
4. On the next frame, the helper sends `NotifyScreenDisabled` upwards before deactivation.
5. The controller finishes cleanup and restores the map after releasing its block.

Important notes

When no Animator/controller is available, closing finishes immediately. A configured controller without the expected trigger/event can leave closing unfinished; the code has no close timeout. Inspect parent/target wiring and verify Back repeatedly on-device. `MobileFramePacing` configures Android's 60 FPS target; it is not proof that a device sustains it. Notification permission denial must not be mistaken for failed reward accounting.

17 Cloud Contract and Ownership

Overview

`firestore.rules` defines permissions and allowed document transitions. Client adapters must satisfy the same accounting and ownership constraints as the game models. Loading Firebase successfully does not validate a deployment or establish server connectivity.

Collection reference

Path under database documents	Role and access intent
users/{uid}/stepStreams/{streamId}, activity/main	Continuous cursors and aggregate activity
users/{uid}/wallet/main	Owner wallet and validated accounting changes
users/{uid}/pointReceipts, spendReceipts, milestoneReceipts	Identified rewards, spends and free-roll claims
users/{uid}/dog/main, dogTrainingReceipts	Owner stats and paired training receipts
users/{uid}/inventory/main	Saved unlock IDs/equipment
users/{uid}/walks, sharedRoutes	Owner data with conditional follower read access
leaderboards/allTime/players/{uid}	Aggregates readable by signed-in players
leaderboardProfiles, friendCodes, friendCodeOwners, friendCodeLookup	Profile and permanent-code data with path-specific access
friends/{uid}/links/{otherId}	Requested/accepted friendship
socialProfiles and social mirrored indexes	Sharing setting and directional following
raceRooms/{code}	Exact-code room access and validated entrant/host operations

Transaction boundaries

Continuous-step credit commits its related cursor, aggregate, wallet and score together. Training commits debit, dog and receipt together. Walk-summary synchronization and shared-route upload are separate paths. Gacha debit/claim and item grant are also separate; transaction guarantees from training must not be attributed to that flow.

Change procedure

1. Identify the model constants, adapter payload and rule paths affected by a change.
2. Update the client and rules together where they enforce one gameplay contract.
3. Exercise allowed and denied transitions with emulator tests.
4. Check compatibility with previously distributed clients and saved data.
5. Record the actual backend deployment and build verification separately.

Dependencies and notes

The functions directory contains server source and emulator tests. Its presence does not establish current deployed functions. The root Firebase configuration points to repository rules and a Firestore emulator. Do not describe this manual as confirmation that production rules match this commit. Client-derived movement remains an input trust limitation even when accounting writes are constrained.

18 Verification and Handover Checklist

Existing checks

Unity tests under `Assets/Tests/Editor` cover tracking, persistence, territory, continuous steps, wallets, racing, rankings, friends, social features and maps. `functions/test/leaderboard.test.js` contains Firebase tests. `scripts/test-map-routes.cjs` exercises map rendering behavior. Prior feature notes report historical results; this manual adds no new passing test claim.

Commands and execution

Use Unity Test Runner for the project's EditMode tests. With the declared Node.js dependencies installed, `npm run test:emulator` from `functions` starts the configured Firestore emulator around its tests; `npm test` alone is not equivalent to that environment. From the repository root, `node scripts/test-map-routes.cjs` runs the map checks. Record tool versions and actual outputs for the build being handed over.

Acceptance record

Check	Evidence to record	Current status
Authentication and account switch	Sign-in, cancellation and stale-request outcomes	Not executed for this document
Continuous/offline steps	Local totals, credited wallet and repeat-sync behavior	Not executed for this document
Walk/recovery/territory	Route gaps, recovery ownership and saved claim	Not executed for this document
Purchase and training	Balance, item/stat result and failure handling	Not executed for this document
Two-phone room race	Shared roster/seed and matching finish order	Not executed for this document
Following privacy	Sharing on/off and follow/unfollow read outcomes	Not executed for this document
Animated navigation	Completion callbacks and restored map	Not executed for this document

Screenshots to add from the actual project

1. Build settings and gameplay scene hierarchy.
2. Persistent manager/bootstrap Inspector and map RectTransform.
3. Gacha catalog and inventory card prefab assignments.
4. Training stat rows, race lanes, lobby controls and close Animator event wiring.
5. Walk summary/history route, training before/after and collar inventory.
6. Room roster/results on two phones, profile sharing controls and error/recovery screens.

Capture date, revision, device and a useful caption for each image. Actual Inspector and runtime images are still to be captured for the reviewed build.

19 Source Index and Maintenance

Source locations

Paths are relative to the repository root. Main C# gameplay sources are under `Walking Dog/Assets/Scripts`.

System	Source location
Project configuration	Walking Dog/ProjectSettings/ProjectVersion.txt and EditorBuildSettings.asset; Packages/manifest.json inside the Unity project
Authentication	Walking Dog/Assets/Firebase/FirebaseLoginManager.cs, LoginUIManager.cs, AndroidGoogleSignIn.cs
Steps and walks	Manager/StepCountAndGpsManager.cs; StepCounterDisplay.cs; GPSTracker.cs; WalkGpsFilter.cs; AndroidWalkTracking.cs
Persistence	Persistence/ContinuousSteps.cs, LocalWalkRepository.cs, WalkSyncService.cs, WalkCloudContracts.cs, FirebaseWalkBootstrap.cs
History/recovery	WalkHistoryUI.cs; WalkRecoveryUI.cs; Persistence/WalkHistory.cs and FirebaseWalkHistoryStore.cs
Territory	TerritoryCapture.cs; TerritoryGeometry.cs; TerritoryService.cs; Persistence/LocalTerritoryRepository.cs
Map	Map/OpenFreeMapWebViewMap.cs; UI/MapSuspender.cs; Walking Dog/MAP_SETUP.md
Wallet	Persistence/PointsWallet.cs, FirebasePointsWalletStore.cs and FirebaseContinuousStepsStore.cs
Gacha/inventory	Manager/GachaManager.cs, CollarInventoryUI.cs and GachaInventoryCardUI.cs; Persistence/CollarInventoryStore.cs; ScriptableObj/GachaObject/GachaItem.cs
Dog/racing	Racing/DogStats.cs, FirebaseDogTrainingStore.cs, DogRaceSimulation.cs, RaceLobbyModels.cs, FirebaseRaceLobbyService.cs and DogGameScreens.cs
Rankings/social	Leaderboard directory, including FirebaseLeaderboardService partial files, FirebaseLeaderboardWriter, SocialRouteSync and UI controllers
Animations/native	Walking Dog/Assets/animation/DisableanimatedObject.cs and feature subfolders; Assets/Plugins/Android Java files
Backend/tests	firestore.rules; firebase.json; functions/package.json; functions/test; scripts; Walking Dog/Assets/Tests/Editor

Documentation maintenance

When a system changes, update its method/state tables, flow, saved values, affected rule contract and verification evidence together. Keep stable item IDs and data-version behavior explicit. Avoid renaming serialized fields without handling existing scene/prefab data. Dispose listeners and preserve account checks when adding asynchronous operations.

The existing `docs` feature notes retain deeper implementation and dated rollout history. `docs/game-documentation.md` contains the broader game overview and player guide. This system manual is the main developer handover following the supplied reference's organization.